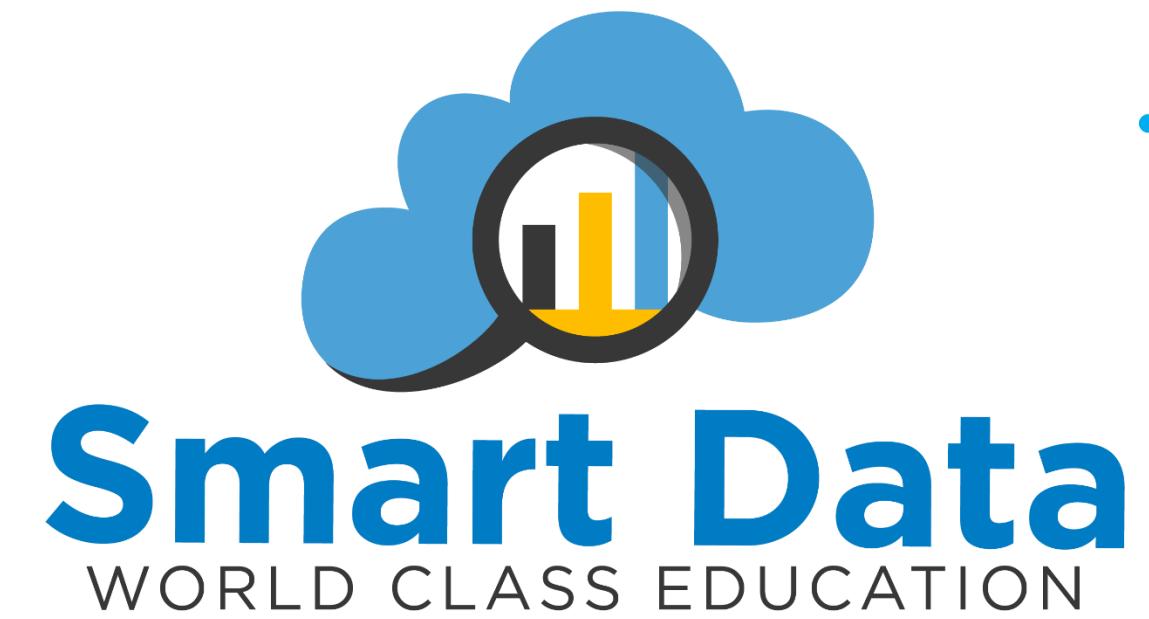
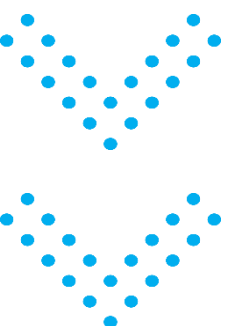
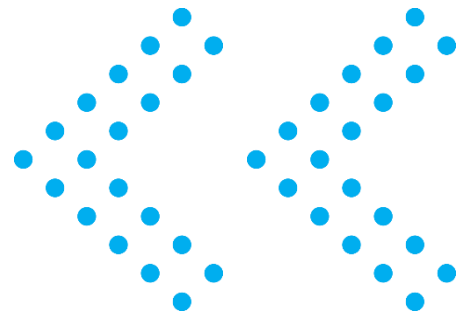
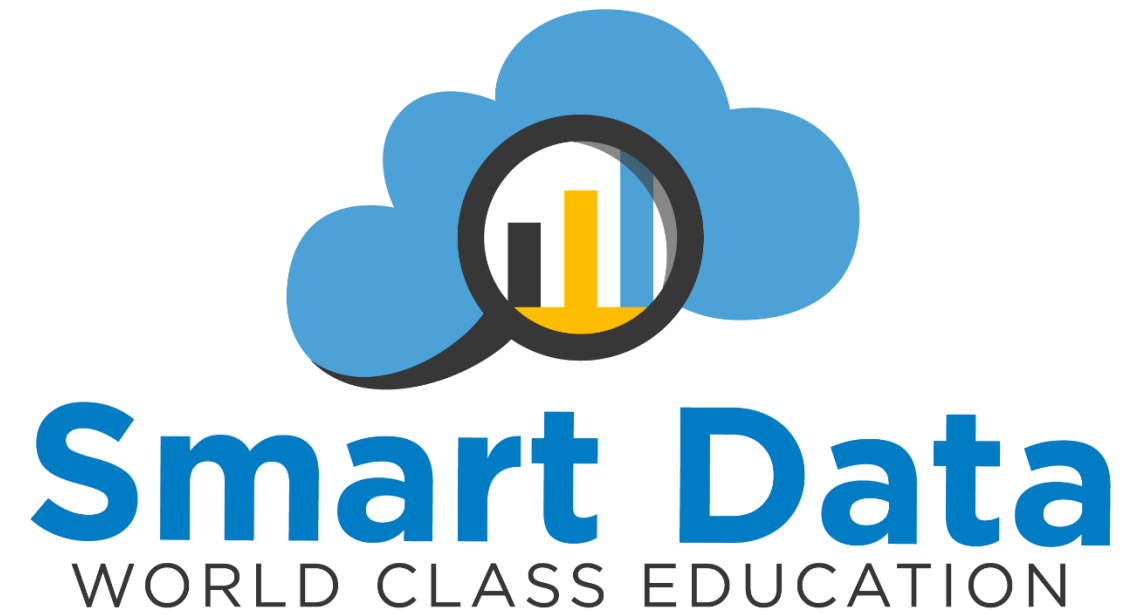




MLOps: Del Modelo al entorno Productivo





Monitoreo y mantenimiento

Instructor:

Julio César Bernal Fernández

Agenda

01

¿Por qué los modelos se degradan?

Model Decay & Drift

02

Data Drift vs. Concept Drift

Tipos de degradación

03

Introducción a EvidentlyAI

Instalación y primeros pasos

04

Métricas de Monitoreo Básicas

KPIs de salud del modelo

El Modelo en Producción

Un modelo entrenado hoy puede volverse obsoleto mañana



Smart
Data
Consulting

World Class Education

Entrenamiento

Datos históricos
estáticos

Validación

Métricas offline
acptables

Despliegue

Sirve predicciones
en vivo

Producción

El mundo
cambia...



Datos de entrada cambian

Distribución del input se desplaza. Ej: nuevos clientes, cambios de mercado.



Realidad subyacente cambia

La relación features→target evoluciona. El comportamiento humano muta.



Degradación silenciosa

Sin alertas activas, el modelo sigue prediciendo aunque sea incorrectamente.

¿Qué es el Model Drift?

"Model drift es el fenómeno por el cual el rendimiento de un modelo de ML se degrada a lo largo del tiempo porque los datos del mundo real cambian respecto a los datos de entrenamiento."

~91%

de empresas reporta degradación de modelos en 9 meses o menos

3–6

meses es el ciclo promedio de reentrenamiento recomendado

>60%

de modelos en prod no tienen monitoreo activo

¿Cuándo comienza el drift?

Inmediatamente — desde el momento en que el mundo cambia respecto a los datos de entrenamiento. Puede tardar días o años en ser visible.

Causas Principales del Drift



Cambios en datos de entrada

Nuevas fuentes de datos, cambios de sensores, actualizaciones de pipelines upstream.



Cambio en comportamiento

Usuarios cambian hábitos, mercados evolucionan, regulaciones nuevas.



Estacionalidad no capturada

Modelos entrenados sin datos suficientes de picos/temporadas.



Errores de pipeline

Errores silenciosos en features, encoding incorrecto, valores faltantes inesperados.



Feedback loops

El modelo afecta el comportamiento que predice (ej. precios, recomendaciones).



Eventos externos

Pandemias, crisis económicas, cambios regulatorios abruptos.

Consecuencias del Drift No Detectado

Pérdidas financieras

Decisiones basadas en predicciones erróneas (crédito, pricing, inventory).

Daño a clientes

Recomendaciones incorrectas, diagnósticos médicos fallidos, fraudes no detectados.

Erosión de confianza

Stakeholders pierden confianza en el equipo de ML.

Costos de remediación

Reentrenar tarde cuesta más: nuevos datos, etiquetado, ciclos de deploy.



Smart
Data
Consulting

World Class Education

Caso Real

COVID-19 destruyó modelos de demanda, fraude y crédito en días. Empresas sin monitoreo tardaron meses en detectarlo. Las que tenían alertas actuaron en semanas.

McKinsey, 2020

<https://www.mckinsey.com/capabilities/quantumblack/our-insights/covid-19-and-the-need-for-real-time-ml-monitoring>

Taxonomía del Drift

DATA DRIFT

$P(X)$ cambia

La distribución de los datos de entrada (features) cambia, pero la relación entre X e Y permanece igual.

Ejemplos:

- Edad promedio de clientes aumenta
- Nuevos países de origen
- Cambio en resolución de imágenes
- Valores monetarios cambian por inflación

CONCEPT DRIFT

$P(Y|X)$ cambia

La relación fundamental entre features y el target cambia. Lo que antes predecía bien ahora es incorrecto.

Ejemplos:

- Cambio en el significado de 'fraude'
- Nuevos patrones de compra post-pandemia
- Actualización de regulaciones financieras
- Cambio de preferencias del usuario

Tipos de Concept Drift

Súbito (Abrupt)

Cambio drástico e inmediato. Ej: lanzamiento de regulación, pandemia.

Gradual

Cambio progresivo a lo largo del tiempo. Ej: evolución de hábitos de consumo.

Incremental

Pequeños cambios acumulados. Difícil de detectar. Requiere ventanas largas.

Recurrente

El concepto cambia y vuelve. Ej: estacionalidad, comportamiento semanal.

💡 *La estrategia de detección varía según el tipo de drift. No existe un único detector universal.*

Métodos de Detección de Drift

1

Statistical Tests

Kolmogorov-Smirnov, Chi-Square, PSI (Population Stability Index), Wasserstein Distance.

2

Model-Based

Entrenar un clasificador para distinguir datos de train vs. producción.

3

Window Methods

Page-Hinkley, ADWIN, DDM. Detectan cambios en streams de datos.

4

Performance Drop

Monitorear métricas como Accuracy, F1, AUC directamente en producción.

PSI — Population Stability Index (más usado en industria)

PSI < 0.1 → Sin cambio. **0.1 – 0.25** → Cambio moderado, investigar. **> 0.25** → Drift significativo, acción requerida.

Data Drift en la Práctica

Ejemplo: Modelo de scoring crediticio



Feature	Distribución Train	Distribución Prod	PSI	Estado
Edad	$\mu=38, \sigma=10$	$\mu=45, \sigma=12$	0.31	Drift alto
Ingresos	$\mu=2500$	$\mu=2800$	0.14	Investigar
Historial	Cat. balanceado	Cat. sesgado	0.38	Drift alto
Meses cliente	$\mu=18$	$\mu=21$	0.08	Estable
Score previo	$\mu=650$	$\mu=640$	0.06	Estable

¿Qué es EvidentlyAI?

Evidently es una librería open-source de Python para evaluar, testear y monitorear modelos de ML en producción. Genera reportes interactivos y dashboards sin necesidad de infraestructura adicional.



Reportes HTML

Reportes visuales listos para compartir con stakeholders.



Test Suite

Validación automática con umbrales configurables.



Monitoring Suite

Dashboard de monitoreo continuo con snapshots.



Integración

Compatible con MLflow, Grafana, Airflow, Prefect.



Smart
Data
Consulting

World Class Education

Por qué Evidently

- ✓ Open-source (Apache 2.0)
- ✓ 100% local — sin enviar datos a la nube
- ✓ +20 métricas de drift incluidas
- ✓ HTML reports + JSON para pipelines
- ✓ UI web con Evidently Cloud opcional
- ✓ Gran comunidad y docs activas

```
pip install evidently
```


Instalación y Setup Local

01 Instalar Evidently

```
pip install evidently
```

Python 3.7+ recomendado. Incluye pandas, numpy, scipy, plotly.

02 Importar librerías

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset
from evidently.metrics import *
```

Los 'presets' agrupan métricas comunes para casos de uso típicos.

03 Preparar datos

```
reference = df_train # datos históricos
current   = df_prod  # datos en producción
```

Ambos DataFrames deben tener las mismas columnas.

04 Generar reporte

```
report = Report(metrics=[DataDriftPreset()])
report.run(reference_data=reference,
           current_data=current)
report.save_html('drift_report.html')
```

Abre el HTML en el navegador para ver el análisis interactivo.

Tipos de Reportes y Presets

DataDriftPreset

Detecta si la distribución de features ha cambiado entre referencia y producción.

DataQualityPreset

Analiza calidad: nulos, duplicados, outliers, tipos de datos.

RegressionPreset

Métricas de regresión: MAE, RMSE, R^2 , error por segmento.

ClassificationPreset

Accuracy, F1, Precision, Recall, ROC-AUC, matriz de confusión.

TargetDriftPreset

Analiza si la distribución del target ha cambiado.

TextOverviewPreset

Para modelos NLP: analiza distribución y calidad de textos.

Ejemplo Completo: Detección de Drift

```
import pandas as pd
from sklearn.datasets import load_iris
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, DataQualityPreset
from evidently.metrics import ColumnDriftMetric

# Cargar datos
iris = load_iris(as_frame=True)
df = iris.frame
reference = df.sample(frac=0.6, random_state=42)
current = df.sample(frac=0.4, random_state=99)

# Reporte con múltiples presets
report = Report(metrics=[
    DataDriftPreset(),
    DataQualityPreset(),
    ColumnDriftMetric(column_name='sepal length (cm)'),
])
report.run(reference_data=reference, current_data=current)
report.save_html('iris_drift_report.html')
```

Output generado

-  Resumen de drift por columna
-  Histogramas comparativos
-  Estadísticos: p-value, drift score
-  Visualización interactiva (Plotly)
-  JSON para integración en pipelines
-  HTML compatible con el equipo

Test Suites: Validación Automática

Los Test Suites permiten definir condiciones de aceptación y recibir PASS/FAIL automático.

```
from evidently.test_suite import TestSuite
from evidently.tests import *

suite = TestSuite(tests=[
    TestNumberOfColumnsWithMissingValues(lt=2),
    TestShareOfDriftedColumns(lt=0.3),
    TestColumnDrift('age', stattest='ks',
                    stattest_threshold=0.05),
    TestColumnDrift('income'),
    TestNumberOfOutliers(lt=10),
])

suite.run(reference_data=ref, current_data=cur)
suite.save_html('test_results.html')

# Acceder resultados programáticamente
results = suite.as_dict()
passed = results['summary']['all_passed']
```

💡 *Ideal para CI/CD de ML: incluir en pipelines de Airflow, GitHub Actions o MLflow.*

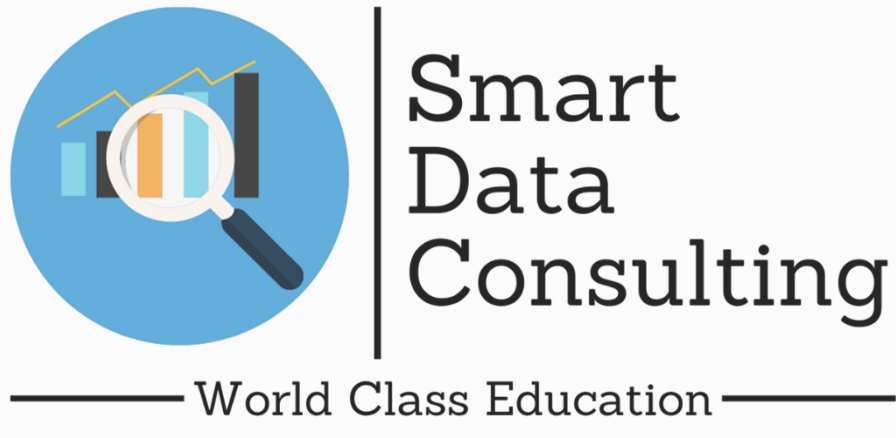
✓ **PASS**

Todos los tests superan el umbral definido. El modelo puede desplegarse.

✗ **FAIL**

Al menos un test falló. Se bloquea el deploy automático y se genera alerta.

Métricas de Performance del Modelo



CLASIFICACIÓN		
Accuracy	$TP+TN / N$	Funciona bien solo con clases balanceadas.
Precision	$TP / (TP+FP)$	¿Qué % de predicciones positivas son correctas?
Recall	$TP / (TP+FN)$	¿Qué % de positivos reales detectamos?
F1 Score	$2 \cdot P \cdot R / (P+R)$	Balance entre Precision y Recall.
ROC-AUC	$\int \text{ROC curve}$	Capacidad discriminativa global del modelo.

REGRESIÓN		
MAE	$\text{mean } y - \hat{y} $	Error absoluto promedio. Fácil de interpretar.
RMSE	$\sqrt{\text{mean } (y - \hat{y})^2}$	Penaliza errores grandes más que MAE.
MAPE	$\text{mean } y - \hat{y} / y \cdot 100\%$	Error porcentual. Útil para series de tiempo.
R²	$1 - SS_{\text{res}} / SS_{\text{tot}}$	Varianza explicada. Cercano a 1 es mejor.
ME	$\text{mean } (y - \hat{y})$	Error medio. Detecta sesgos sistemáticos.

Métricas de Calidad de Datos

Compleitud

- % valores nulos por columna
- Conteo de registros
- % columnas con >X% nulos

Distribución

- Media y desviación estándar
- Percentiles (p5, p25, p75, p95)
- Valores únicos / Cardinalidad

Consistencia

- Tipos de datos por columna
- Valores fuera de rango
- Duplicados

Correlaciones

- Correlación entre features
- Feature importance shift
- Correlación con target

Dashboard de Monitoreo Continuo

Arquitectura de monitoreo en producción con Evidently



```
from evidently.ui.workspace import Workspace
from evidently.ui.dashboards import DashboardConfig

ws = Workspace('monitoring_workspace')
project = ws.create_project('Credit Scoring')

# Ejecutar cada hora via cron
report = Report(metrics=[
    DataDriftPreset(),
    ClassificationPreset(),
])
report.run(reference_data=ref, current_data=cur)
ws.add_run(project.id, report)

# Ver dashboard: evidently ui --workspace .
```

Alertas sugeridas

- Drift en >30% de features
- PSI > 0.2 en feature clave
- AUC cae > 5% vs baseline
- Tasa de nulos > 2x baseline
- Nuevas categorías desconocidas

KPIs Avanzados de Monitoreo



Prediction Drift

Monitorear la distribución de las predicciones mismas, no solo el input. Si el modelo predice siempre la misma clase, hay un problema.

```
KS test sobre  $P(\hat{y}_{ref})$  vs  $P(\hat{y}_{cur})$ 
```



MTTR y MTTD

Mean Time To Detect (MTTD) y Mean Time To Remediate (MTTR). Métricas de eficiencia operativa del equipo de ML.

```
MTTD = tiempo promedio hasta alerta
```



Model Staleness

Días desde el último reentrenamiento. Umbral aceptable depende del dominio: banca (30d), e-commerce (7d), ads (1d).

```
staleness = now() - last_train_date
```



Coverage

% de predicciones que caen dentro del dominio de entrenamiento. Predicciones fuera del dominio son menos confiables.

```
coverage = IN_DOMAIN / TOTAL
```



Calibration

¿Las probabilidades predichas reflejan la frecuencia real? Un modelo bien calibrado: 70% de confianza → 70% de correctas.

```
 $ECE = \sum |conf - acc| \cdot n/N$ 
```



Data Freshness

¿Con qué retraso llegan los datos? Si los features tienen lag, el modelo usa información obsoleta en cada predicción.

```
lag = max(now() - data_timestamp)
```

Recursos y Referencias

Documentación Oficial

Evidently AI Docs

<https://docs.evidentlyai.com/>

Evidently GitHub

<https://github.com/evidentlyai/evidently>

All Metrics Reference

<https://docs.evidentlyai.com/reference/all-metrics>

Artículos Clave

ML Monitoring — Chip Huyen

<https://huyenchip.com/2022/02/07/data-distribution-shifts-and-monitoring.html>

Concept Drift Survey (arXiv)

<https://arxiv.org/abs/2004.05785>

MLOps Principles

<https://ml-ops.org/content/mlops-principles>

Herramientas Complementarias

Grafana — Dashboards

<https://grafana.com/>

MLflow — Experiment Tracking

<https://mlflow.org/>

Neptune.ai — ML Monitoring

<https://neptune.ai/>

Lecturas Adicionales

Google ML Crash Course

<https://developers.google.com/machine-learning/crash-course>

Interpretable ML Book

<https://christophm.github.io/interpretable-ml-book/>

Data Drift Blog — EvidentlyAI

<https://www.evidentlyai.com/blog/machine-learning-monitoring-data-and-concept-drift>

Conclusiones Clave



Los modelos SIEMPRE se degradan

No es una posibilidad, es una certeza. La pregunta es cuándo y cuánto.



Data drift $\neq$ Concept drift

La causa determina la solución. No todo drift requiere reentrenamiento.



Evidently: simple y poderoso

100% local, open-source, con reportes listos en minutos.



Monitorear = prevenir

Un sistema de alertas temprano ahorra semanas de remediation.